

The Orthogonalized Read Is a Removable Training Scaffold for Recurrent Memory

Keston Aquino-Michaels
No Way Labs

July 2026

Abstract

A recent report finds that orthogonalizing the mLSTM memory matrix at read time (five Newton–Schulz iterations, trained through) substantially improves noisy associative recall [23]. The effect replicates, but it is not a memory improvement. Training on this task is a long chance plateau followed by a sharp escape, and the orthogonalized read acts by re-conditioning the learning problem during the plateau. Three properties establish this. It must be *self-consistent*: an exact recursive least-squares read (the Mesa layer) reproduces it, while straight-through halves, delta-rule writes, frozen random keys, and plain normalization all fail. It is *uniform*: across a learning-rate $\times$ hardness grid it multiplies the escape hazard roughly six-fold with no detectable hardness dependence, widening the workable learning-rate corridor that narrows for the baseline. And it is *removable*: applied to failed models at inference it rescues none, and annealed away on an escape-triggered schedule it leaves numerically stock mLSTMs at full accuracy. Much of the published gain needs no architecture at all — solved-rate at a fixed budget measures escape hazard, which follows a heat/noise law (learning-rate elasticity +3.0, gradient-noise elasticity −1.65) under which the original vocab-96 result is a large-batch noise condition rather than a capacity one. Decoding the memory state directly shows failed models carry roughly half their associations in linearly recoverable form: the plateau is a readout failure over half-written storage. Two conclusions travel beyond the intervention: recall benchmarks used for architecture selection partly measure trainability, and the system is a fully instrumented model organism of “emergence,” in which a sharp behavioral threshold demonstrably arises from a censored metric over gradually accumulating structure.

1 Introduction

Precise associative recall from context is the capability that separates softmax attention from recurrent sequence models: the perplexity gap between the two concentrates on recall-heavy tokens [2, 3], and the modern linear-recurrence program—mLSTM/xLSTM [4], delta-rule fast weights [22, 29], selective state spaces [8], RWKV [18]—is largely an attempt to make a fixed-size matrix memory fake the behavior of attention’s unbounded KV cache on exactly this capability. Synthetic recall suites such as MAD [19] and Zoology [2] exist precisely to *select* among these architectures cheaply, at small scale, on the premise that the ranking transfers: this is the explicit “mechanistic design” methodology [19]. A model’s solved-rate on the benchmark, at a fixed training budget, is read as its recall *capability*.

This paper studies one intervention on one such benchmark and finds that the solved-rate is measuring something else. On MAD noisy recall, training is a long chance plateau followed by a sharp escape, so solved-rate at a fixed budget is a censored measurement of *escape hazard*—how readily the optimizer leaves the plateau—not of asymptotic capability. This matters beyond the

intervention, because the property being selected for (small-scale, small-budget trainability) is exactly the one least likely to survive the change in schedule, budget, and gradient-noise regime that accompanies scaling—the transfer that mechanistic design assumes. It also mirrors, in miniature and fully instrumented, the standing critique of emergent-capability claims in large models: sharp thresholds on a metric, measured at a few censored compute budgets, over structure that in fact accumulates gradually [21, 28].

The intervention we study is from Tambde [23] (henceforth “the original study”), which reports that a one-block mLSTM’s performance on MAD noisy associative recall improves dramatically when the memory matrix $\mathbf{C}_t$ is orthogonalized at read time by five Newton–Schulz iterations [6, 10], with gradients flowing through the iteration. That study motivates the fix by recall needs in attention-free settings such as long-horizon model-based RL [9]; we take the empirical claim on its own terms. It is a good test case precisely because it looks like a clean architectural win: a cheap fix for the core weakness of recurrent memory, interference in a superposed matrix store.

This paper asks what that intervention actually does, and answers with the mechanism of the title: the orthogonalized read is a removable training scaffold. It does not add memory capacity; it re-conditions the learning problem the optimizer sees during a specific phase of training, and once training is done it can be taken down. Concretely, our contributions:

1. **Replication** (§3). At the setup of Tambde [23] the effect reproduces: the orthogonalized read (NS5) roughly triples the fraction of seeds that solve noisy recall (9/16 vs. 3/16; pooled over two regimes 19/32 vs. 7/32, Fisher $p = 0.005$).
2. **The schedule confound** (§4). Training is a chance-plateau $\rightarrow$ sharp-escape process, and solved-rate at a fixed budget is a censored measurement of escape time. Stretching the cosine period from 2,000 to 4,000 steps lifts the *baseline* from 3/16 to 11/16 solved *at step 2,000*—the same training compute (McNemar $p \approx 0.008$). A constant learning rate alone reaches 8/16 — we cannot distinguish it from NS5 (log-rank $p = 0.41$; hazard ratio 1.5, 95% CI [0.6, 4.3]) — at one sixth of NS5’s per-step cost. Per unit wall-clock the schedule levers dominate NS5 at this hardness — a wall-matched constant-LR baseline solves 14/16 in the time NS5 needs for 9/16 (Fig. 3); at the v96 frontier the ordering plausibly reverses (§7).
3. **Mechanism by elimination** (§5). The residual benefit requires self-consistent whitening of the read. An exact Sherman–Morrison/RLS read matches NS5; both straight-through halves (whitened forward with raw gradients, and vice versa) land at 0/8; delta-rule erase-before-write keeps the memory perfectly conditioned (effective rank 9.9 vs. 2.6) and helps nothing; frozen random keys and read-path learning-rate boosts change nothing; Frobenius normalization without the whitening iterations is baseline (1/16). Memory rank collapse is a symptom of the plateau, not its cause.
4. **The scaffold property** (§6). Read-time orthogonalization rescues no failed baseline from chance at inference; 7/9 solved NS5 models keep solving with the orthogonalized read removed (a 5–9 point tax); and annealing the read back to raw over the last 30% of training closes the tax: 12/16 seeds finish as *numerically stock mLSTMs* at 95–100% accuracy. Gating the anneal on escape rather than a fixed window recovers the late escaper: 13/16 solved — matching the un-annealed combination exactly — with every solver finishing at $\alpha=0$, mean accuracy 99.6% on the raw read.
5. **Where the scaffold matters** (§7). On a 144-run grid (three constant learning rates $\times$ three vocabulary sizes $\times$ two variants $\times$ eight seeds), the whitened read multiplies the escape hazard

by $6.4\times$ (95% CI [3.5, 11.7]; no detectable hardness interaction), rescuing the too-cold edge of the learning-rate corridor and suppressing destabilization at the too-hot edge. The baseline’s hazard shrinks with hardness, so the *realized* gap grows even though the multiplier is flat.

6. **The escape law** (§8). Across a batch-size $\times$ learning-rate grid, escape hazard rises with per-step heat (elasticity 3.0, 95% CI [2.0, 4.1]) and falls with SGD gradient noise lr^2/B (elasticity -1.65 , $[-2.1, -1.2]$); their cancellation explains the learning-rate corridor, and batch size moves the corridor because it enters only the noise term. Reproducing the original study’s headline regime (vocab 96, batch 64, its own schedule), the baseline goes from all-chance to marginal (1/8) and the orthogonalized read to 5/8 — a +45-point mean gain matching the original report, which we attribute to the escape-hazard multiplier acting on a baseline that large batch has lifted to the edge of escape. The headline regime required large batch as a *noise* condition, not a capacity one.
7. **Storage/readout dissociation** (§9). Ridge-decoding the memory state directly: behaviorally failed models ($\sim 3\%$ accuracy) carry $\sim 52\%$ linearly decodable, key-specific associations (wrong-key control $\sim 5\%$). The plateau is a readout-learning failure over half-written storage, which is why write-side conditioning cannot fix it.

Two consequences extend beyond the intervention and are developed in §11. First, recall benchmarks used for architecture selection partly measure trainability: when the underlying quantity is escape hazard, a small-scale ranking reflects optimization ease as much as capability, and the audit protocol used here (escape-time survival analysis, schedule and batch robustness checks, storage/readout probes) applies directly to that practice. Second, the system provides a fully instrumented example of an “emergent” threshold: capability-relevant structure is present and decodable while the behavioral metric reads zero, and the apparent discontinuity can be traced to the metric and the optimizer rather than to the representation.

2 Setup

Task. MAD noisy in-context recall [19]: sequences of key $\rightarrow$ value token pairs with 80% interleaved noise tokens drawn from a disjoint distractor range; at each query position the model must emit the value previously paired with the queried key. Vocabulary 80 (key/value ranges) unless stated, sequence length 512, accuracy measured only at query positions. We vendor the MAD generator; data are generated fresh each batch from a seeded stream.

Model. One-block mLSTM [4]: embedding dim 94, 4 heads, pre/post LayerNorm, $\sim 78\text{K}$ parameters, exactly the reference implementation of Tambde [23] (verified against the author’s code). Per head, the mLSTM maintains a matrix memory $\mathbf{C}_t = \sum_{u \leq t} w_{t,u} \frac{\mathbf{k}_u}{\sqrt{d_h}} \mathbf{v}_u^\top$ with gate-determined weights $w_{t,u}$, read as $\mathbf{h}_t = \mathbf{q}_t^\top \mathbf{C}_t / \max(|\mathbf{q}_t^\top \mathbf{n}_t|, e^{-m_t})$.

The intervention. NS5 replaces the read by $\mathbf{q}_t^\top \text{NS}_5(\mathbf{C}_t)$, where NS5 is Frobenius normalization followed by five Newton–Schulz iterations toward the nearest orthogonal matrix, applied per position and trained through. Its per-position materialization of $\mathbf{C}_t$ makes it the compute driver: $\sim 6\times$ baseline wall time at our batch size, and $O(b s d_h^2)$ activation memory.

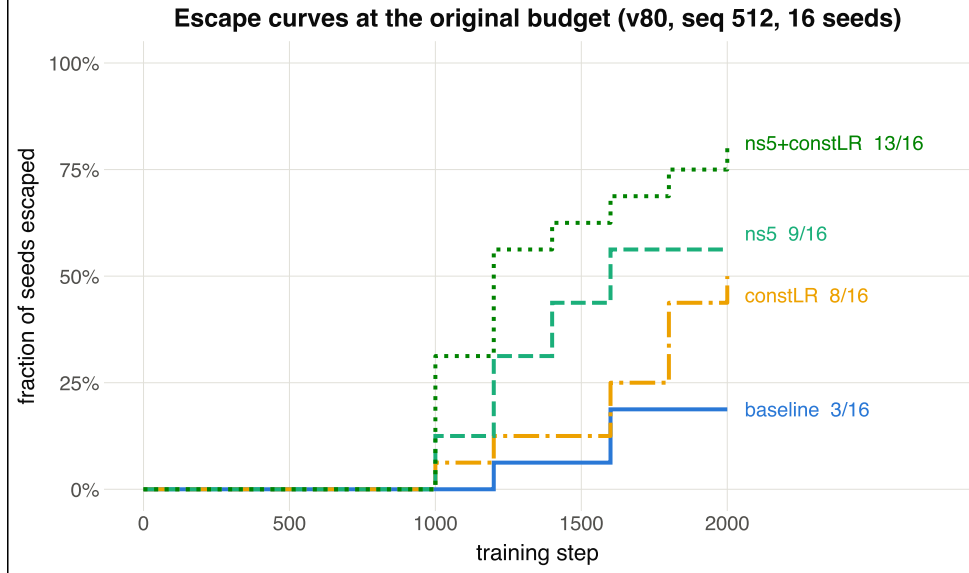


Figure 1: Escape curves at the original budget of Tambde [23] (v80, seq 512, cosine $T_{\max}=2000$ except where noted; 16 seeds each). The orthogonalized read (NS5) triples the solved fraction over the baseline — but deleting the LR decay (constLR) recovers most of that gap for free, and the two levers together (NS5+constLR) dominate. Curves are Kaplan–Meier estimates of the fraction of seeds escaped by each step.

Variants. Mechanism probes used throughout (`memrec/models.py`): NSk (with k iterations; NS0 = normalization only), NS5_FWD/NS5_BWD (straight-through halves), DELTA (erase-before-write, $\beta=1$), RLS (Sherman–Morrison whitened read $\mathbf{q}^\top (\sum \hat{\mathbf{k}}\hat{\mathbf{k}}^\top + \lambda I)^{-1} \mathbf{C}$ — the Mesa-layer read [24, 25]), RANDK (frozen random-orthogonal key projections), POGO_QK (q/k projections kept orthogonal by POGO [12]), and read-path learning-rate boosts. All variants share modules and training loop; state dictates cross-load, which is what enables the swap evaluations of §6.

Training and metrics. AdamW [15], lr 3×10^{-3} , batch 16, 2,000 steps with cosine decay ($T_{\max}$ = budget) unless stated — the regime of Tambde [23]. Outcomes per seed are *bimodal*: a run either stays at chance ($\sim 3\%$) or escapes sharply to $\geq 90\%$. We therefore never report mean accuracy. The primary metric is the escape-time distribution (first evaluation with accuracy $\geq 80\%$), analyzed with Kaplan–Meier estimates [14], log-rank tests [16] with runs censored at budget, and discrete-time hazard models [1]; solved-fraction tables (Fisher exact, McNemar for seed-paired comparisons) are reported alongside. Batch 16 (vs. 64 in the original study) keeps every regime a fair A/B under identical compute; §8 treats batch size explicitly. Two contrasts are designated primary and confirmatory: NS5 vs. baseline escape hazard at the original schedule, and the seed-paired schedule-stretch flip (McNemar). All other comparisons are exploratory; we report their unadjusted p -values as graded evidence and draw no individual confirmatory claim from them, and we accompany null results with effect-size confidence intervals rather than equivalence claims.

3 Replication

In the regime of Tambde [23] our reimplementaion reproduces the direction and rough size of the published effect (Fig. 1): NS5 solves 9/16 seeds vs. the baseline’s 3/16 at v80/s512, 10/16 vs. 4/16

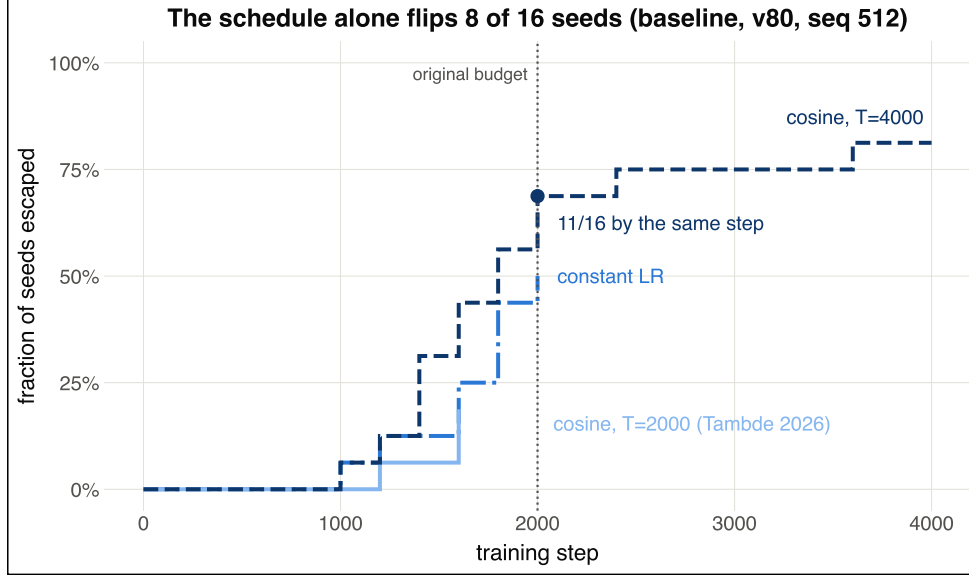


Figure 2: The schedule alone flips half the seeds. Baseline mLSTM, v80/s512, same 16 seeds per curve. Stretching the cosine period to $T_{\max}=4000$ lifts the baseline from 3/16 to **11/16 solved by step 2,000** — identical training compute, McNemar $p \approx 0.008$. Escape requires the learning rate to still be hot when the run is ready to transition; aggressive decay strangles it.

at v80/s1024 (pooled 19/32 vs. 7/32, Fisher $p = 0.005$; log-rank on escape times $p = 0.020$). The author’s code reproduces the same direction at batch 16 at vocab 80. The original headline vocab-96 regime does not reproduce at batch 16 under its own schedule—every variant collapses to chance; the original needed batch 64 on an A100. We return to both facts (§7, §8): they are the first hints that the benchmark is measuring an optimization quantity.

4 The schedule confound: solved-rate is censored escape time

Training on this task sits at chance for hundreds to thousands of steps, then transitions sharply (Fig. 6 shows example traces). The shape is familiar from grokking [20], with one difference in kind: classic grokking is delayed *generalization* after the training set has been memorized, whereas this plateau shows no task learning at all before the discovery; the escape-dynamics analysis carries over (§8). A fixed step budget with a decaying learning rate therefore *censors* the escape process twice: runs that would escape later are cut off by the budget, and—more insidiously—the decay itself removes the optimizer heat that escape requires.

Fig. 2 isolates the effect. With the cosine period stretched from $T_{\max}=2,000$ to 4,000 (same seeds), the baseline solves **11/16 by step 2,000**—the very step count at which the original schedule yields 3/16—and 13/16 by step 4,000. Eight seeds flip failed→solved from the schedule alone (McNemar $p \approx 0.008$). The same pattern holds at s1024 (4/16 → 10/16 → 15/16). Deleting the decay entirely (constant LR, stop at 2,000) yields 8/16 with five more seeds mid-escape (57–70%) at cutoff. We cannot distinguish this from NS5 at the original budget: log-rank $p = 0.41$, and a two-group discrete-time fit puts the NS5-vs-constant-LR hazard ratio at 1.5 with 95% CI [0.6, 4.3] — 16 seeds per arm leave effects up to $\sim 4\times$ in either direction compatible, so this is a statement of non-detection, not equivalence. What 16 seeds *do* establish is the per-compute ordering at this hardness: NS5’s per-position Newton–Schulz graph costs $\sim 6\times$ baseline wall time per step, and

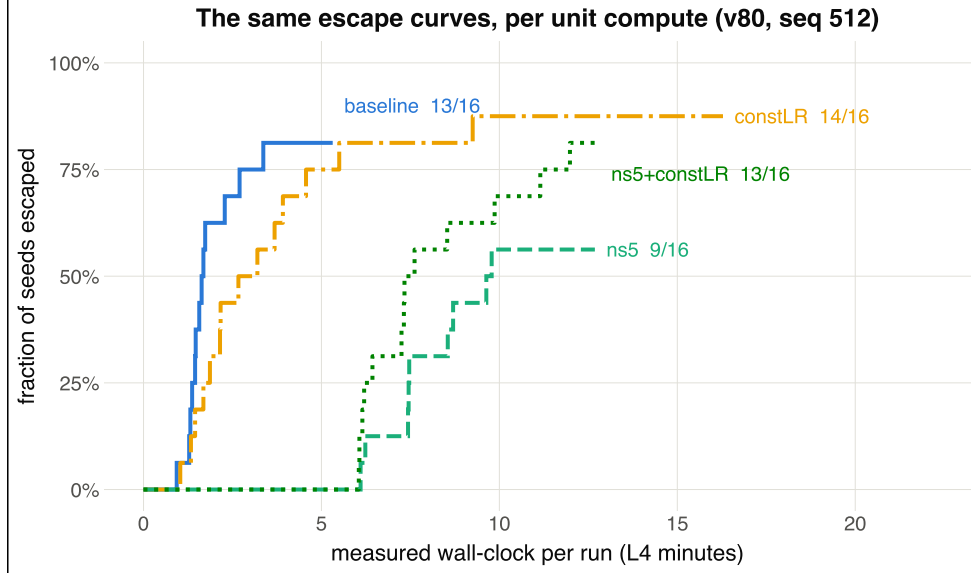


Figure 3: The escape comparison per unit compute, plotted against each run’s *measured* wall-clock on identical L4 GPUs. Each family appears under its strongest known schedule recipe — baseline under cosine $T_{\max}=4000$ and under constant LR with a wall-matched 12,000-step budget; NS5 under its published schedule and with the constant-LR lever added — because the practical question is what to run under a fixed wall-clock budget; pairing NS5 against weaker baseline schedules would only flatter it. Given the ~ 12 minutes NS5 needs to solve 9/16, the wall-matched constant-LR baseline solves 14/16, most of them within its first four minutes.

a wall-matched arm makes the consequence concrete — given NS5’s ~ 12 -minute budget (12,000 constant-LR steps), the plain baseline solves **14/16**, above NS5’s 9/16 and NS5+constLR’s 13/16 at the same wall cost (Fig. 3). At the v96 frontier, where no tested budget makes the baseline reliable, the per-compute ordering plausibly reverses (§7).

NS5 retains a real residual edge at matched schedules: 16/16 at $T_{\max}=4,000$ including all three seeds the baseline never cracks, with systematically earlier escapes (median step 1,200 vs. 1,600; log-rank $p = 0.031$). The rest of the paper is about that residual: what it is (§5), that it is removable (§6), and where it matters (§7).

5 Mechanism by elimination: self-consistent whitening of the read

During the plateau, the memory matrix is noise-dominated: 80% of writes are distractor tokens, and $\mathbf{C}$ ’s effective rank collapses from ~ 11 at initialization to ~ 2.5 within 200 steps. The matched-filter read $\mathbf{q}^\top \mathbf{C}$ over such a matrix defines an ill-conditioned learning problem for the read path (q/k projections): gradient coherence between independent batches, measured on the q/k projections during the plateau, is 0.20 for the baseline vs. 0.42 for NS5 — the whitened read *doubles* read-path gradient signal-to-noise while full-model coherence is identical (0.44 vs. 0.42).

Table 1 tests every adjacent explanation at the original schedule.

Five readings:

Dissociation kills the state-conditioning story. The delta-rule write never lets $\mathbf{C}$ rank-collapse (effective rank 9.9 vs. the baseline’s 2.6, Fig. 4) and solves nothing (2/16). The rank collapse observed in failing baselines is what not-learning looks like, not why learning fails.

variant	what it changes	solved	q/k grad-coherence	C eff. rank
baseline	—	3/16	0.20	2.6
DELTA (erase-before-write)	state conditioning	2/16	0.21	9.9
RANDK (frozen random keys)	measurement ensemble	3/16	—	—
q/k LR $\times 4$	optimizer pressure on read path	4/16	—	—
NS5_FWD (NS fwd, raw grads)	forward only	0/8	—	—
NS5_BWD (raw fwd, NS grads)	backward only	0/8	—	—
NS0 (normalize, 0 iters)	scale only, no whitening	1/16	—	—
NS1 (one NS iteration)	partial whitening	0/16	0.27	3.3
NS3 (three NS iterations)	most of the whitening	6/16	0.41	3.8
NS5 [23]	self-consistent spectral whitening	9/16	0.42	4.2
RLS (Sherman–Morrison read)	self-consistent exact whitening	5/8	0.28	4.3
baseline, constant LR	schedule only	8/16	0.45 / 0.20 [†]	—
NS5 + constant LR	both levers	13/16	—	—

Table 1: The elimination table. All cells: v80/s512, cosine $T_{\max}$ =2000 unless noted, 16 seeds (8 for RLS and straight-through cells). Plateau metrics averaged over steps 200–600, before any escape. [†]escapers/non-escapers at step 600 within constant-LR baseline runs.

Keys are fine everywhere. The key-Gram effective rank is indistinguishable across variants and outcomes (≈ 5.5 – 5.8); frozen random-orthogonal keys change nothing. Recall failure here is not a degraded measurement matrix, contra a compressed-sensing intuition.

It is the full whitening, not normalization or a nudge toward it (Fig. 5). The dose response is monotone above $k=1$ and thresholded. NS0 (Frobenius normalization, zero iterations) is baseline (1/16). A single Newton–Schulz iteration lands *below* the baseline (0/16 vs. 3/16), with plateau read-path coherence (0.27) only slightly above the baseline’s 0.20. The samples are too small to establish the dip (Fisher $p = 0.23$; log-rank $p = 0.07$), but its direction matches the straight-through cells: a partially whitened read, like an inconsistently whitened one, can hand the optimizer a worse problem than the raw read, not a fraction of a better one. Three iterations recover nearly all of the coherence (0.41 vs. NS5’s 0.42) and most of the benefit (6/16 vs. 9/16) at 70% of NS5’s wall time (mean 8.5 vs. 12.3 minutes per run, baseline 2.0); the escape statistics of NS3 sit between baseline and NS5 and do not separate from either at these sample sizes. The practical cost floor of the read is therefore near $k=3$, and mild spectrum shaping below that buys nothing.

No transplantable half. Both straight-through cells are at or below baseline (0/16 pooled; decisively worse than NS5, Fisher $p \approx 8 \times 10^{-4}$). Whitened gradients applied to a raw forward mis-describe the read that actually happened; a whitened forward with raw gradients mis-assigns credit. The benefit lives in the agreement: the whitened read defines a different, better-conditioned optimization problem, and the optimizer must actually optimize *that* problem. This is why we call it self-consistent whitening.

The exact decoder confirms the family. RLS is not an ad-hoc probe: the recurrently computed regularized least-squares read is exactly the Mesa layer of von Oswald et al. [24], scaled up in MesaNet [25], and the space it spans — raw, delta-rule, and least-squares reads as choices of regressor and optimizer — is the one formalized by test-time regression [26], of which Newton–Schulz is a spectral approximation and the delta rule an online-gradient one. It matches or exceeds NS5 (5/8, earlier escapes, log-rank vs. baseline $p = 0.022$, vs. NS5 $p = 0.92$) in a recurrent $O(d_h^2)$ /step form: the known locally optimal read architecture lands in the same whitening family our elimination isolates.

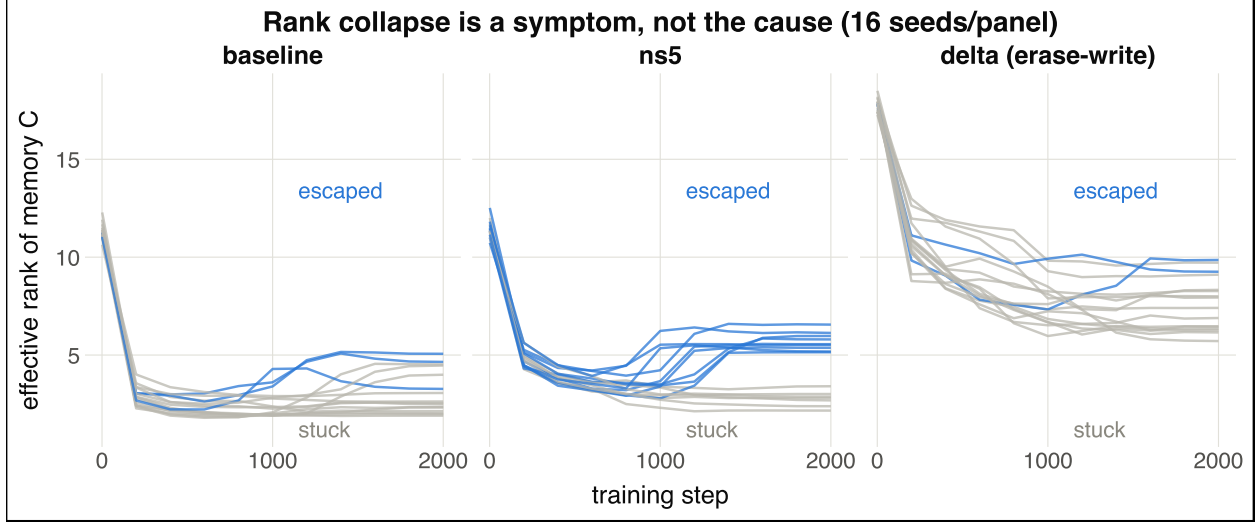


Figure 4: Memory rank collapse is a symptom, not the cause. Effective rank of the raw memory $\mathbf{C}$ (final position, first validation batch) over training; one line per seed, colored by outcome. Baseline and NS5 escapes coincide with rank recovery — but DELTA’s erase-before-write holds effective rank ~ 10 throughout and still fails (2/16). Conditioning the *state* is not the lever; conditioning the *read during learning* is.

Finally, read-path gradient coherence is a *thermometer*, *not a thermostat*: pre-escape coherence discriminates future escapers within every route measured (within constant-LR baseline runs: escapers 0.45 vs. non-escapers 0.20 at step 600), but it is not directly settable—boosting the read-path learning rate $4\times$ does nothing (4/16), and no plateau probe at a fixed early step predicts *when* a seed escapes (all $p > 0.2$). Elevated coherence is the shared signature of runs on their way out; the demonstrated levers are self-consistent read whitening and schedule heat.

6 A removable scaffold

If the whitened read works by re-conditioning the learning problem, it should be unnecessary once learning is done. Three tests confirm this.

Swap evaluations. Re-evaluating trained checkpoints under the other read: bolting NS₅ onto failed baselines at inference rescues **none from chance** (12/12 at-chance failures stay at chance; the single borderline seed, at 78%, edges to 82%) — the benefit is not read-time denoising. Conversely **7/9** solved NS₅ models keep solving with the orthogonalized read removed (mean 97%→88%; a consistent 5–9 point tax), and NS₅ on solved baselines is a +1.6 point polish.

Scaffold anneal. Interpolating the read $\alpha \text{NS}_5(\mathbf{C}) + (1 - \alpha) \mathbf{C}$ from $\alpha=1$ to 0 over the final 30% of training (with constant LR) closes the tax entirely: **12/16 seeds finish at 95.4–99.8% (mean 98.4%) with $\alpha=0$** — the final artifact is numerically a stock mLSTM with zero inference overhead (Fig. 6). Cost vs. the un-annealed combination (13/16): a single late-escaping seed caught mid-anneal by the fixed window.

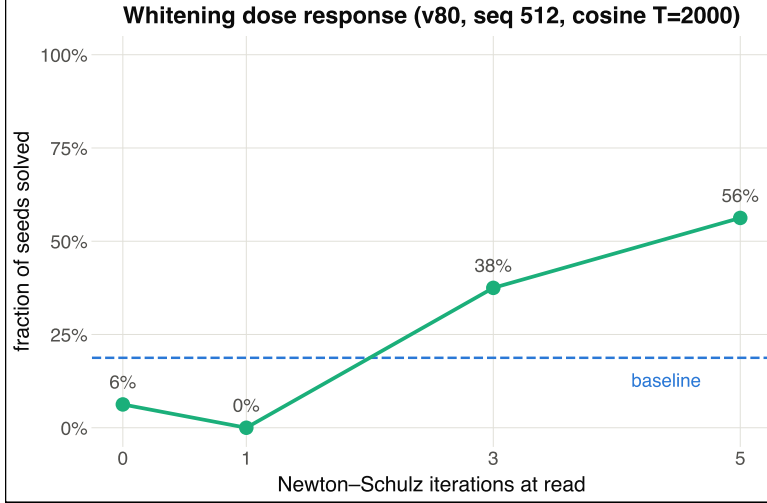


Figure 5: Whitening dose response: fraction of seeds solved vs. Newton-Schulz iteration count at the read (16 seeds per point, original schedule; dashed line = baseline mLSTM at 3/16). Normalization alone ($k=0$) and a single iteration confer nothing; three iterations recover nearly all of the read-path gradient coherence and most of the benefit at 70% of NS5’s wall time.

Escape-gated anneal. Triggering the anneal on the first evaluation $\geq 80\%$ (200-step dwell, 500-step ramp, budget extended when needed so late escapers finish their ramp) removes the fixed-window failure mode. The gate fires only *after* a run has already escaped within the nominal budget, so the extension cannot manufacture solves — it exists solely to complete the scaffold’s removal, and given §4 we flag it explicitly: any accuracy gained during extension steps is post-escape polish at the raw read, not escape. Sixteen seeds under the gate (constant LR, 200-step dwell, 500-step ramp): **13/16 solved, all thirteen at $\alpha=0$, mean 99.6%** (min 99.3%) on the raw read — the un-annealed combination’s solve rate, now with zero inference overhead and no accuracy tax. Trigger steps ranged from 1,000 to 2,000; nine runs extended their budget (at most +900 steps, all post-escape) to complete the ramp. The three failures never escaped and are the same class of seed the un-annealed combination loses.

The recipe also holds where it matters most. At the v96 frontier (constant LR, 4,000-step budget, the regime where the scaffold’s advantage is largest), the escape-gated anneal solves **7/8, all seven finishing at $\alpha=0$ on the raw read** at 97.6–100% (mean 98.6%). The full pipeline — scaffold, escape, removal — therefore delivers a numerically stock mLSTM at frontier accuracy, not only at the easier v80 regime.

Train with the scaffold; take it down before deployment.

7 Where the scaffold matters: the basin map

The elimination table pins *what* the residual effect is; the basin map pins *where it matters*. We ran constant LR $\{10^{-3}, 3 \times 10^{-3}, 10^{-2}\} \times$ vocab $\{80, 88, 96\}$ at seq $768 \times \{\text{baseline}, \text{NS5}\} \times 8$ seeds $\times 3,000$ steps (Fig. 7). Four readings:

The baseline’s corridor narrows with hardness. v80 solves at two of three learning rates; v88 at exactly one (10^{-3} drops to 0/8); v96 at none fully (best 3/8). Hardness does not merely lower the solve rate—it shrinks the set of hyperparameters at which solving is possible at all.

Whitening widens the corridor in both directions. Cold edge: v88 at 10^{-3} goes 0/8 $\rightarrow$

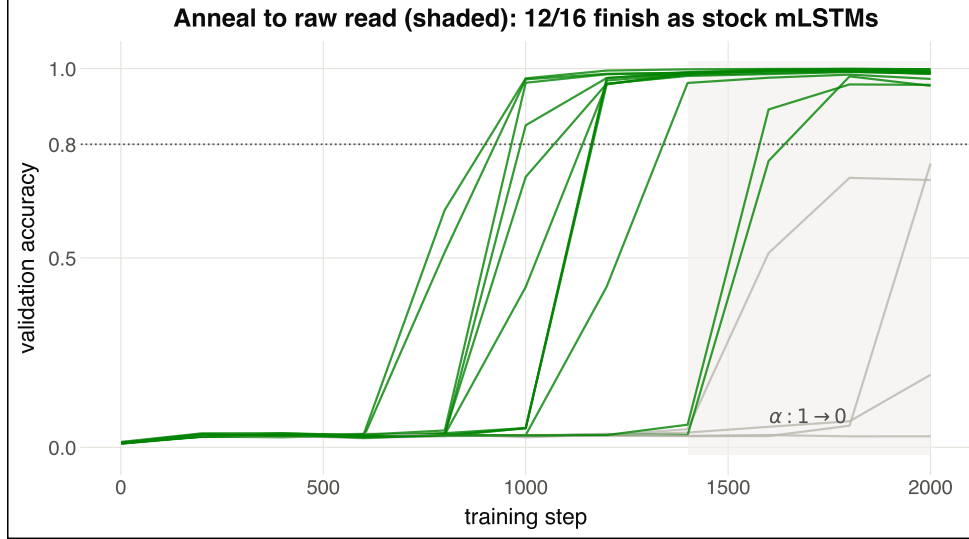


Figure 6: Scaffold anneal (fixed window, shaded): the orthogonalized read is interpolated back to the raw read over steps 1400–2000. Twelve of 16 seeds end the run as numerically stock mLSTMs at 95–100% accuracy. The gray late escaper caught mid-anneal motivates the escape-gated variant.

6/8 (log-rank $p = 0.0025$). Hot edge: baselines *destabilize* at 10^{-2} (19/24 collapse below chance across regimes) while NS5 at v80 has zero collapses and solves 5/8 — though this stability protection fades with hardness (dead: 0/8 $\rightarrow$ 2/8 $\rightarrow$ 4/8). Center: NS5 is 24/24 at 3×10^{-3} across all vocabulary sizes vs. the baseline’s 14/24.

A uniform multiplier under a shrinking base rate. A discrete-time hazard model (1,633 run-intervals, 70 escapes; destabilized runs censored at collapse as a competing risk) gives NS5 a $6.4\times$ escape-hazard multiplier (95% CI [3.5, 11.7]). Adding $\text{NS5} \times \text{hardness}$ interaction terms yields nothing ($\text{NS5} \times \text{v88}$: HR 1.03, 95% CI [0.29, 3.7]; $\text{NS5} \times \text{v96}$: 0.74, [0.18, 3.1]) — consistent with a hardness-independent multiplier, though with 70 escape events the intervals leave factor-of-three modulation in either direction compatible. The baseline hazard itself falls with hardness (v96: $0.14\times$, $z = -5.5$). A flat multiplier on a shrinking base produces the growing realized advantage — including the headline vocab-96 regime of Tambde [23]. (An earlier fit on cosine-schedule data suggested the multiplier itself grew with hardness; the constant-LR grid shows that “interaction” was a censoring artifact of the schedule.)

The learning-rate effect is non-monotone. Relative to 10^{-3} , hazard at 3×10^{-3} is $4.9\times$ but at 10^{-2} is $0.23\times$ — *plus* the competing destabilization risk. A pure cumulative-heat law ($\int \text{lr } dt$) fails: v80 escapes at 10^{-3} on *less* total heat than 3×10^{-3} escapes use, while the cold column at v88/v96 is still at chance or mid-climb at budget (censoring, one level up). The natural suspect for the missing term is SGD noise ($\propto \text{lr}^2/B$), which motivates the batch axis.

At matched constant-LR schedules the frontier regime shows the practical consequence: at v96/s768 with 4,000 steps, the baseline manages 1/4 solved (plus two mid-escapes) while NS5 solves 4/4 — the original headline regime, reached at roughly 1/8 of its original compute. Note the contrast with Fig. 3: here even the per-compute comparison plausibly favors the whitened read (4/4 at $\sim 6\times$ per-step cost, against a baseline no tested budget makes reliable), though we have not run a wall-matched baseline arm at this regime. The realized advantage grows with hardness exactly as the uniform-multiplier law predicts. The two levers are additive: NS5+constant LR at the original 2,000-step budget is the best step-matched cell on the board (13/16; log-rank vs. constant LR alone

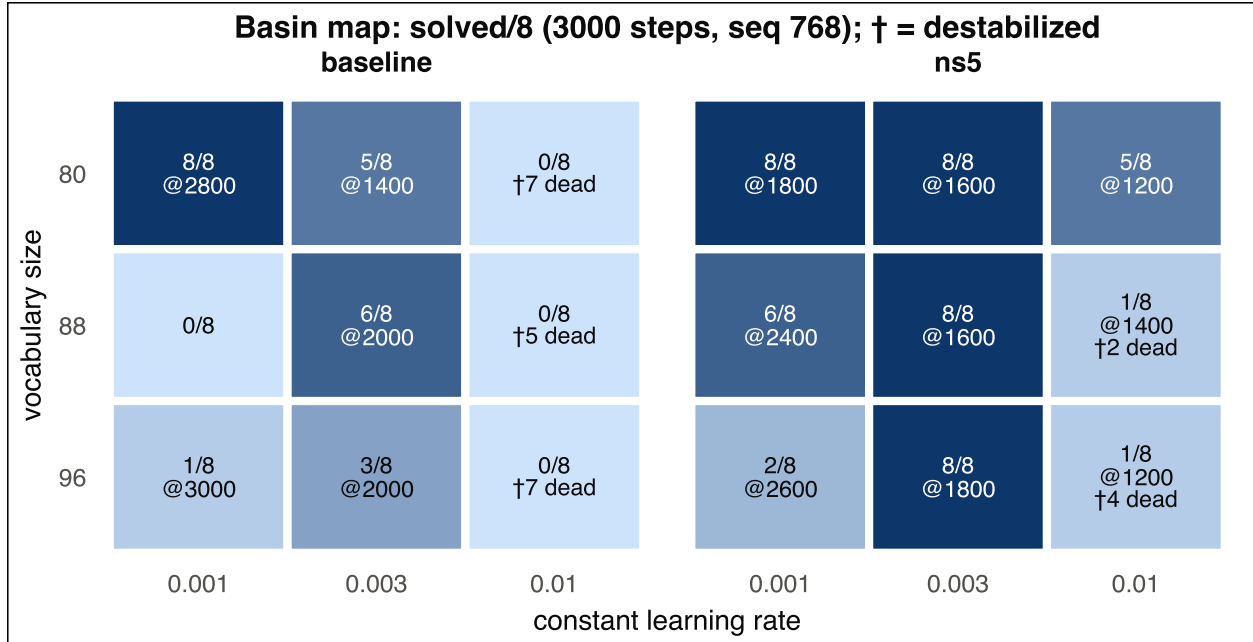


Figure 7: Basin map: solved seeds out of 8 per cell (constant LR $\times$ vocabulary size, seq 768, 3,000 steps), with median escape step and destabilization counts († = accuracy collapses below chance — a competing risk distinct from plateauing). The baseline’s workable-LR corridor narrows as the task hardens; the whitened read widens it in both directions and lifts its center (24/24 at 3×10^{-3}).

$p = 0.017$). Per unit compute the ranking differs (§4, Fig. 3): a wall-matched constant-LR baseline reaches 14/16, since each NS5 step costs $\sim 6\times$ a baseline step.

8 The batch axis: heat vs. noise in the escape law

The basin map varied heat; batch size varies gradient noise at fixed heat. We ran the baseline at batch $\{4, 8, 16, 32, 64\} \times$ constant LR $\{10^{-3}, 3 \times 10^{-3}, 10^{-2}\} \times 8$ seeds (v80/s512; Fig. 8), an NS5 arm at batch $\{4, 8, 16\}$ at the reference LR, and one reconciliation arm at the original study’s own setting.

The corridor moves with batch size. At batch ≤ 16 the workable learning rate is 10^{-3} and 10^{-2} is lethal (23/24 destabilized); at batch 32–64 the corridor’s center shifts to 3×10^{-3} (6/8 and 7/8) and the hot edge becomes survivable. The main study’s batch of 16 — chosen so NS5 fits a 24 GB GPU — therefore sits in a noisier, narrower-corridor regime than the original study’s batch 64, which the reconciliation below makes consequential.

The escape law separates heat from noise. A discrete-time hazard fit over the 120 baseline runs (1,645 run-intervals, 45 escapes; runs censored at collapse or their own budget) with terms $\log t$, $\log \text{lr}$, and $\log B$ rejects both simple laws: a pure cumulative-heat law predicts no batch effect ($\log B$ coefficient: 1.65, $z = 6.8$), and a tokens-processed law predicts equal coefficients on $\log B$ and $\log \text{lr}$ (the latter is -0.26 , $z = -1.4$). Refitting the same likelihood in the (heat, noise) basis — $\log \text{lr}$ at fixed noise, and $\log(\text{lr}^2/B)$, the standard SGD gradient-noise scale — gives a clean decomposition: escape hazard rises with per-step heat with elasticity 3.0 (95% CI [2.0, 4.1]) and falls with gradient noise with elasticity -1.65 ($[-2.1, -1.2]$). The near-null raw learning-rate coefficient is their cancellation ($3.04 - 2 \times 1.65 = -0.26$): raising the LR buys heat linearly in log-space but

Baseline corridor across batch size (v80, seq 512)				
batch size (budget)	64 (2000 steps)	6/8	7/8	2/8 †4 dead
	32 (2000 steps)	0/8	6/8	2/8 †1 dead
	16 (3000 steps)	5/8	3/8	0/8 †7 dead
	8 (4000 steps)	4/8	2/8	0/8 †8 dead
	4 (6000 steps)	6/8	2/8	0/8 †8 dead
		0.001	0.003	0.01
		constant learning rate		

Figure 8: Baseline solved fractions across batch size $\times$ constant LR (v80/s512, 8 seeds per cell). Budgets scale inversely with batch (printed per row) so that small-batch cells are not trivially censored; the hazard model below accounts for the differing budgets through censoring. The workable corridor sits at 10^{-3} for batch ≤ 16 and moves to 3×10^{-3} at batch 32–64; 10^{-2} destabilizes 23/24 runs at batch ≤ 16 but becomes survivable at batch 64.

noise quadratically. This *is* the corridor of §7 — its cold edge is heat-starvation, its hot edge is noise plus the destabilization risk — and batch size relocates it because B enters the noise term alone. Concurrent theory arrives at the same variables from the other direction: Ersoy and Wiesner [7] model grokking as noise-driven escape from a metastable phase and derive escape-time scaling in B/η for linear networks; our fit is the empirical decomposition on a recall task, and neither result subsumes the other.

The whitened read’s multiplier survives at the noisy end. At the reference LR, NS5 vs. baseline by batch: 6/8 vs. 2/8 at batch 4 (log-rank $p = 0.049$), 7/8 vs. 2/8 at batch 8 ($p = 0.006$), 13/16 vs. 8/16 at batch 16 ($p = 0.017$). The scaffold rescues exactly the cells that shrinking batch abandons, consistent with the uniform-multiplier reading of §7.

The original vocab-96 result was a batch requirement, and batch is noise. We reproduce the original study’s headline regime and split it. Under its own cosine schedule at v96/s768, batch 64 takes the *baseline* from all-chance (batch 16, §3) to marginal: 1/8 solved, 2/8 mid-escape, mean 22%. Adding the orthogonalized read at that same setting gives 5/8 solved, mean 67% — a +45-point mean gain, matching the original study’s reported +40-point delta at vocab 96. The gain is real, but it is the scaffold’s $\sim 6\times$ escape-hazard multiplier (§7) acting on a baseline that large batch has already lifted to the edge of escape; nothing about capacity is needed. The batch-64 requirement was a gradient-noise requirement, and the headline gain rode on it.

9 Inside a failed model’s memory

If the plateau were a storage failure, write-side fixes should help; they do not (Table 1). We probed storage directly: ridge-decode the value token from the feature $\mathbf{k}_u^\top \mathbf{C}_t$ — the model’s *own* stored key at the pair’s write position, slicing its *own* memory at query time — fit on a train split of held-out validation sequences and scored on unseen sequences, with a wrong-key control (the same pipeline keyed by a different stored pair from the same sequence).

group	behavioral acc	C-decode (correct key)	C-decode (wrong key)
failed baselines ($n=13$)	$\sim 3\text{--}14\%$	52.3%	5.2%
failed NS5 ($n=7$)	$\sim 3\%$	52.3%	4.8%
solved baselines ($n=3$)	93.3%	88.4%	5.0%
solved NS5 ($n=9$)	97.1%	98.3%	5.0%

Table 2: Ridge decode of the raw memory state (32 checkpoints from the replication sweep; features $\mathbf{k}_u^\top \mathbf{C}_t$ at the pair’s write position; held-out sequences). Failed models’ memories are half-written and fully unread.

Failed models’ memories are half-written and fully unread: models at chance carry $\sim 52\%$ linearly decodable, key-specific associations they cannot behaviorally retrieve (Table 2). The plateau is a readout-learning failure over partially intact storage; escape co-develops both (and NS5-trained solutions end with cleaner storage than baseline ones, 98% vs. 88% decodable). This explains the delta-rule null—storage was never the binding constraint. Two controls sharpen the picture: the decode succeeds only with the key from the pair’s *value* position (where the causal convolution has folded key and value together before the write); and a naive oracle-query substitution ($\mathbf{q}_t := \mathbf{k}_u$ inside the full forward) fails its positive control — solved models drop from 93–97% to 19–40% — so the working readout is not literal key-matching; the learned query geometry is essential.

10 Practical guidance

- **Fix the schedule before adding machinery.** On plateau-escape tasks, deleting the LR decay reproduced the entire published gain at the original study’s own budget, at 1/6 the per-step cost — and per unit wall-clock the schedule levers dominate the orthogonalized read at this hardness (Fig. 3). Cosine-to-budget couples the schedule to the budget and silently censors escape.
- **If you whiten, whiten self-consistently — then take it down.** Straight-through variants are worse than nothing. Train with NS_k or an RLS read, anneal to the raw read late in training (gate the anneal on escape), and deploy a stock mLSTM. RLS obtains the same benefit in a linear-cost recurrent form if per-position NS is too expensive.
- **Audit recall-benchmark claims for censoring.** Solved-rate at one budget/schedule measures escape hazard, not capability. Report escape-time survival curves, vary the schedule, and separate destabilization from plateauing as competing risks. The diagnostic probes here (state-spectrum traces, read-path gradient coherence) are cheap and trajectory-neutral.

11 Discussion

Benchmark-driven architecture selection. MAD and Zoology were built to compare candidate architectures cheaply at small scale, on the premise that small-scale rankings predict behavior at scale [2, 19]. Our results complicate that premise. On this benchmark, solved-rate at a fixed budget is a censored measurement of escape hazard (§4), and an apparent architectural advantage can be produced by the learning-rate schedule alone; in the case studied here the schedule accounts for most of the published effect. To the extent that other recall benchmarks share the plateau-escape training dynamics, a ranking taken at a single budget and schedule confounds capability with small-scale

trainability. Trainability is also the property most sensitive to the changes that come with scale: §7–8 show the workable-hyperparameter region moving with learning rate, hardness, and batch size, so a variant selected for easy escape at 78K parameters and 2,000 steps has no particular claim to the same advantage under a production training recipe. The checks used in this paper are inexpensive and could be adopted as standard practice: escape-time survival curves in place of solved-rate at one budget, replication of the ranking under a second schedule and batch size, destabilization treated as a competing risk rather than folded into failure, and, where checkpoints allow, a probe for stored-but-unread structure (§9).

Relation to emergent-ability claims. The same measurement problem appears, at much larger scale, in the debate over emergent abilities. Wei et al. [28] describe abilities that appear abruptly as compute grows; Schaeffer et al. [21] respond that many such thresholds are produced by discontinuous metrics applied to smoothly improving models. That debate has mostly been conducted through external scaling curves, because the internal state of the models in question is difficult to instrument. The system studied here is small enough to instrument completely, and it realizes the scenario of Schaeffer et al. [21] in a directly checkable form: the behavioral metric sits at chance for thousands of steps while roughly half of the task-relevant associations are already linearly decodable from the memory state (§9), and the eventual jump is an optimization event in the readout pathway (§5) rather than the appearance of new stored structure. We would not argue that every reported emergent ability has this character. What the small system offers is that the question becomes decidable: plateau-escape recall tasks are a setting in which a claimed discontinuity can be tested against the internal state instead of inferred from the metric.

Scaffolded training. The recipe that comes out of §6 — optimize through a better-conditioned reparameterization of the read, then anneal the reparameterization away before deployment — has relatives elsewhere: quantization-aware training inserts the quantizer during training and removes it afterwards [11], and batch-norm folding and auxiliary losses likewise discard training-time structure at inference. Two observations from this study seem relevant to that broader family. First, the whitening helps only when the forward and backward computations agree (§5), which suggests self-consistency as a requirement for scaffolds of this type. Second, the connection to optimizer design is close: Muon applies the same Newton–Schulz orthogonalization to gradient updates [13, 27], where it also improves conditioning without changing the model class. Whether other conditioning problems admit removable scaffolds of this kind is a question we leave open; this study contributes one worked example.

12 Related work

Recall as the discriminating capability. Zoology and Based [2, 3] identified precise recall as where the attention-vs-recurrence gap concentrates; MAD [19] systematized synthetic proxies. Write-rule design — delta-rule fast weights [22, 29], gated matrix memories [4], selective state spaces [8] — is largely contention management for a fixed-size store. Wang et al. [26] formalize this design space as test-time regression, with linear attention, the delta rule, and recursive least squares arising as choices of regressor and optimizer; our elimination table is an empirical probe of that taxonomy under training dynamics. Its axes do sort the reads — delta’s online gradient descent and RLS’s exact solve are different test-time optimizers, and they behave differently in our table — but what predicts *trainability* of this task is not any single axis; it is whether the read is a self-consistent whitening (NS, RLS) as opposed to a raw, normalized-only, or straight-through read. Our results

also bear on how this program uses its benchmarks (§11): at the small scale where architecture selection happens, MAD/Zoology solved-rates are substantially optimization-contaminated, so a delta credited to a write rule or read fix may be trainability, not capacity. Schaeffer et al. [21] make the parallel argument about emergent-ability claims from discontinuous metrics; our storage/readout dissociation (§9) supplies the internal-state version.

Plateau–escape dynamics. The chance-plateau / sharp-escape shape, its learning-rate and schedule sensitivity, and the role of gradient noise are shared with the grokking literature [20]. The settings differ in kind — grokking is delayed generalization after train-set memorization, while our plateau precedes any task learning — but the escape analysis transfers, and Ersoy and Wiesner [7] derive a noise-driven escape law in the same (η, B) variables our §8 fit measures empirically.

Orthogonalization in learning systems. Newton–Schulz orthogonalization [6, 10] has recently been prominent in *optimizer* design: Muon [5, 13] orthogonalizes gradient updates, and Wang et al. [27] explains its advantage over Adam via more isotropic singular spectra in associative-memory parameters — a weight-space cousin of the state-space re-conditioning we find here. POGO [12] maintains orthogonal parameter matrices at scale; in our tests parameter orthogonality (pogo_qk) does not reproduce the state-whitening benefit, consistent with the mechanism being about the evolving memory state, not the weights. On the architecture side, the Mesa layer and MesaNet [24, 25] deploy the recursive least-squares read our RLS variant implements, and Variational Linear Attention [17] builds a Sherman–Morrison regularized memory with recall gains attributed to interference control; our swap and anneal results suggest a testable reinterpretation of such designs at small scale — conditioned reads of this family may act on trainability rather than capacity, and can be annealed away once training is done.

Emergence and metric artifacts. Wei et al. [28] catalogued sharp capability thresholds in large models; Schaeffer et al. [21] argued many are artifacts of discontinuous metrics over smoothly improving competence. Our §9 is a mechanistic, internally-instrumented instance of that argument on a plateau-escape recall task (§11).

The proposal under study. Tambde [23] motivates read-time orthogonalization by recall needs in attention-free settings such as Dreamer-style world models [9]. Our replication confirms the observation; our dissection relocates its cause from memory capacity to optimization, and shows the intervention is best used as a removable training aid.

13 Limitations

This is a mechanism study of one intervention on one synthetic task family and one small architecture ($\sim 78\text{K}$ -parameter one-block mLSTM), in the regime where the original claim was made. The scaffold story is established here; whether it transfers verbatim to multi-block models, other recall families, or language-model scale is untested. Several results ride on 8–16 seeds per cell; we mitigate with survival analysis and exact tests, and we release every per-seed trace. The hazard model treats destabilization as independent censoring; a joint competing-risks model might refine the hot-edge story. Two secondary negatives are reported as such (out-of-sample failure of probe-based seed triage; the $\text{init} \times \text{data}$ lottery being dominated by their interaction), and the POGO+NS5 stacking cell remains optimizer-confounded.

Reproducibility

All code (task generator vendored, models, training loop, probes, analysis, and figure scripts), per-seed results with full evaluation traces, and probe telemetry are released at <https://github.com/no-way-labs/recurrent-memory-scaffold>. Every number in this paper regenerates from `runs/*.jsonl` via the analysis scripts. Experiments ran mostly on NVIDIA L4s (Modal, ≤ 10 concurrent), with a single vocab-96 batch-64 arm on an A100; total compute cost of the study was approximately \$120. Probes are trajectory-neutral: training with probes enabled is bit-identical to training without.

References

- [1] P. D. Allison. Discrete-time methods for the analysis of event histories. *Sociological Methodology*, 13:61–98, 1982.
- [2] S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, and C. Ré. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.
- [3] S. Arora, S. Eyuboglu, M. Zhang, A. Timalsina, S. Alberti, D. Zinsley, J. Zou, A. Rudra, and C. Ré. Simple linear attention language models balance the recall-throughput tradeoff. *arXiv preprint arXiv:2402.18668*, 2024.
- [4] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. xLSTM: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.
- [5] J. Bernstein and L. Newhouse. Old optimizer, new norm: An anthology. *arXiv preprint arXiv:2409.20325*, 2024.
- [6] Å. Björck and C. Bowie. An iterative algorithm for computing the best estimate of an orthogonal matrix. *SIAM Journal on Numerical Analysis*, 8(2):358–364, 1971.
- [7] I. T. Ersoy and K. Wiesner. Noise-driven escape from metastable phases explains grokking in deep neural networks. *arXiv preprint arXiv:2606.17120*, 2026.
- [8] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [9] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- [10] N. J. Higham. *Functions of Matrices: Theory and Computation*. SIAM, 2008.
- [11] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [12] A. Javaloy and A. Vergari. An embarrassingly simple way to optimize orthogonal matrices at scale. *arXiv preprint arXiv:2602.14656*, 2026.

- [13] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024.
- [14] E. L. Kaplan and P. Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.
- [15] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [16] N. Mantel. Evaluation of survival data and two new rank order statistics arising in its consideration. *Cancer Chemotherapy Reports*, 50(3):163–170, 1966.
- [17] V. Pandey and G. Singh. Variational linear attention: Stable associative memory for long-context transformers. *arXiv preprint arXiv:2605.11196*, 2026.
- [18] B. Peng, E. Alcaide, Q. Anthony, et al. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.
- [19] M. Poli, A. W. Thomas, E. Nguyen, P. Ponnusamy, B. Deiseroth, K. Kersting, T. Suzuki, B. Hie, S. Ermon, C. Ré, C. Zhang, and S. Massaroli. Mechanistic design and scaling of hybrid architectures. *arXiv preprint arXiv:2403.17844*, 2024.
- [20] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- [21] R. Schaeffer, B. Miranda, and S. Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems*, 2023.
- [22] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.
- [23] A. Tambde. Matrix orthogonalization improves memory in recurrent models. Blog post, June 2026. URL <https://ayushtambde.com/blog/matrix-orthogonalization-improves-memory-in-recurrent-models>. Accessed 2026-07-01.
- [24] J. von Oswald, M. Schlegel, A. Meulemans, S. Kobayashi, E. Niklasson, N. Zucchet, N. Scherrer, N. Miller, M. Sandler, B. Agüera y Arcas, M. Vladymyrov, R. Pascanu, and J. Sacramento. Uncovering mesa-optimization algorithms in transformers. *arXiv preprint arXiv:2309.05858*, 2023.
- [25] J. von Oswald, N. Scherrer, S. Kobayashi, L. Versari, S. Yang, S. Mittal, M. Schlegel, K. Maile, Y. Schimpf, O. Sieberling, A. Meulemans, R. A. Saurous, G. Lajoie, C. Frenkel, R. Pascanu, B. Agüera y Arcas, and J. Sacramento. MesaNet: Sequence modeling by locally optimal test-time training. *arXiv preprint arXiv:2506.05233*, 2025.
- [26] K. A. Wang, J. Shi, and E. B. Fox. Test-time regression: A unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025.
- [27] S. Wang, F. Zhang, J. Li, C. Du, C. Du, T. Pang, Z. Yang, M. Hong, and V. Y. F. Tan. Muon outperforms Adam in tail-end associative memory learning. *arXiv preprint arXiv:2509.26030*, 2025.

- [28] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.
- [29] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.